

Q : -----
What is the advantage of writing instance variable(Object properties/ NSF), constructor and non static block inside an abstract class, If we can't create an object for abstract class ?

Yes, We can't create an object for abstract class but still we can write instance variable, constructor and non static block that means abstract class also contains Object properties and behavior but these properties will initialize through **SUB CLASS OBJECT using super()** so we can write object properties, non static block as well as constructor inside an abstract class as shown in the program below.

```
package com.ravi.asbtract_demo;

abstract class Animal
{
    protected String name;

    public Animal(String name)
    {
        super();
        this.name = name;
    }

    {
        IO.println("NSB");
    }

    public abstract void roam();
}

class Lion extends Animal
{
    public Lion(String name)
    {
        super(name);
    }

    @Override
    public void roam()
    {
        IO.println(name+" Lion is roaming");
    }
}

class Dog extends Animal
{
    public Dog(String name)
    {
        super(name);
    }

    @Override
    public void roam()
    {
        IO.println(name+" Dog is roaming");
    }
}

public class AbstractDemo3
{
    public static void main(String[] args)
    {
        Animal a = new Lion("Simba");
        a.roam();

        a = new Dog("Tommy");
        a.roam();
    }
}
```

```
package com.ravi.asbtract_demo;

abstract class Animal
{
    protected String name;

    public Animal(String name)
    {
        super();
        this.name = name;
    }

    public abstract void regularHealthCheckup();
}

class Lion extends Animal
{
    public Lion(String name)
    {
        super(name);
    }

    @Override
    public void regularHealthCheckup()
    {
        IO.println(name+" lion is going for Health checkup");
    }
}

class Elephant extends Animal
{
    public Elephant(String name)
    {
        super(name);
    }

    @Override
    public void regularHealthCheckup()
    {
        IO.println(name+" elephant is going for Health checkup");
    }
}

class Bird extends Animal
{
    public Bird(String name)
    {
        super(name);
    }

    @Override
    public void regularHealthCheckup()
    {
        IO.println(name+" bird is going for Health checkup");
    }
}

public class AbstractDemo3
{
    public static void main(String[] args)
    {
        Lion lions[] = { new Lion("Simba"), new Lion("Sher"), new Lion("Kesari") };
        regularHealthCheckupCamp(lions);

        IO.println("-----");

        Elephant elephants[] = {new Elephant("Airavat"), new Elephant("Gajraj")};
        regularHealthCheckupCamp(elephants);

        IO.println("-----");

        Bird birds[] = {new Bird("Parrot"), new Bird("Sparrow"), new Bird("Peacock")};
        regularHealthCheckupCamp(birds);
    }

    public static void regularHealthCheckupCamp(Animal ...animals)
    {
        for(Animal animal : animals)
        {
            animal.regularHealthCheckup();
        }
    }
}
```

WAP to show that all the abstract methods must be overridden in sub classes otherwise sub class will become as an abstract class.

```
package com.ravi.asbtract_demo;

abstract class Alpha
{
    public abstract void show();
    public abstract void demo();
}

abstract class Beta extends Alpha
{
    @Override
    public void show() //demo();
    {
        IO.println("Overriding show method in the Beta class");
    }
}

class Gamma extends Beta
{
    @Override
    public void demo()
    {
        IO.println("Overriding demo method in the Gamma class");
    }
}

public class AbstractDemo4 {

    public static void main(String[] args)
    {
        Gamma g = new Gamma();
        g.show();
        g.demo();
    }
}
```

Nested class OR Inner class in Java :

In java it is possible to define a class (inner class) inside another class (outer class) called nested class OR inner class.

Example :

```
public class Outer
{
    class Inner1 //Non static inner class
    {
    }

    static class Inner2 //Static Nested class
    {
    }
}
```

In the same way, It is also possible to define a class/interface/enum/annotation/record inside another class/interface/enum/annotation and record as shown below.

Programs that describes all different possible combinations :

```
-----
package com.ravi.inner_class;

public class Demo1
{
    class A
    {
    }

    interface B
    {
    }

    enum C
    {
    }

    record D()
    {
    }

    @interface E //Annotation
    {
    }
}

package com.ravi.inner_class;

public interface Demo2
{
    class A
    {
    }

    interface B
    {
    }

    enum C
    {
    }

    record D()
    {
    }

    @interface E //Annotation
    {
    }
}

package com.ravi.inner_class;

public enum Demo3
{
    //enum first line is reserved for Universal constants
    RED, BLACK, BLUE;

    class A
    {
    }

    interface B
    {
    }

    enum C
    {
    }

    record D()
    {
    }

    @interface E //Annotation
    {
    }
}

package com.ravi.inner_class;

public record Demo4()
{
    class A
    {
    }

    interface B
    {
    }

    enum C
    {
    }

    record D()
    {
    }

    @interface E //Annotation
    {
    }
}

package com.ravi.inner_class;

public @interface Demo5
{
    class A
    {
    }

    interface B
    {
    }

    enum C
    {
    }

    record D()
    {
    }

    @interface E //Annotation
    {
    }
}
```